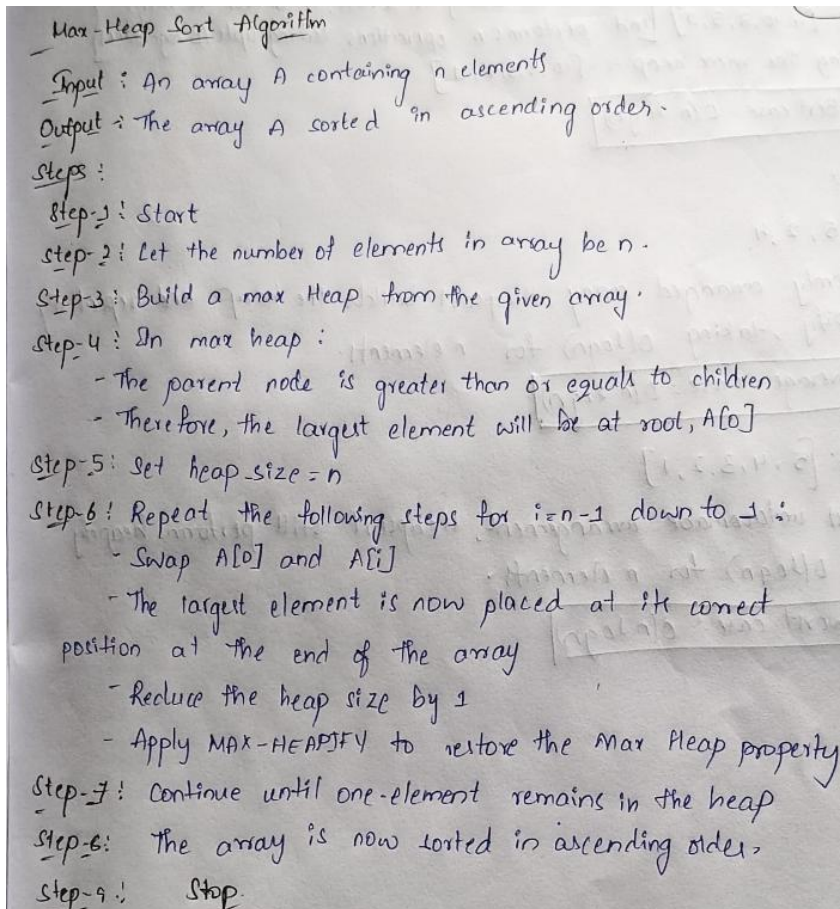


Practical - 3

Aim: To implement the max-heap sort algorithm.

Algorithm :



Max-Heap Sort Algorithm

Input : An array A containing n elements
Output : The array A sorted in ascending order.

Steps :

Step-1 : Start

Step-2 : Let the number of elements in array be n.

Step-3 : Build a max Heap from the given array.

Step-4 : In max heap :

- The parent node is greater than or equals to children
- Therefore, the largest element will be at root, A[0]

Step-5 : Set heap-size = n

Step-6 : Repeat the following steps for i = n-1 down to 1 :

- Swap A[0] and A[i]
- The largest element is now placed at its correct position at the end of the array
- Reduce the heap size by 1
- Apply MAX-HEAPIFY to restore the Max Heap property

Step-7 : Continue until one-element remains in the heap

Step-8 : The array is now sorted in ascending order.

Step-9 : Stop.

Program: #include <iostream>
using namespace std;

```
void heapify(int arr[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;
```



```
    if (largest != i)
    {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n)
{
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--)
    {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    cout << endl;
}

int main()
{
    int arr[] = {12, 11, 13, 5, 6, 7};
    int n = sizeof(arr) / sizeof(arr[0]);

    cout << "Original array: ";
    printArray(arr, n);

    heapSort(arr, n);

    cout << "Sorted array: ";
    printArray(arr, n);

    return 0;
}
```

Output :

```

}
PS C:\Users\vemal\OneDrive\Documents\DSA> g++ -std=c++11 "max heap sort.cpp"
PS C:\Users\vemal\OneDrive\Documents\DSA> .\maxheap.exe
Original array: 12 11 13 5 6 7
Sorted array: 5 6 7 11 12 13
PS C:\Users\vemal\OneDrive\Documents\DSA>

```

Time Complexity:

Time Complexity $\div$ for (int $i = n/2 - 1$; $i \geq 0$; $i--$) $\left\{ \begin{array}{l} \text{heapify(arr, n, i)} \end{array} \right\} (n/2 \cdot \log n)$

for (int $i = n - 1$; $i \geq 0$; $i--$) $\left\{ \begin{array}{l} \text{swap(arr[i], arr[0])} \\ \text{heapify(arr, i, 0)} \end{array} \right\} 1 + \log n$ — $n - 1$

$$T(n) = \frac{n}{2} \log n + (n-1)(1 + \log n)$$

$$T(n) = \frac{n}{2} \log n + (n-1) + (n-1) \log n$$

$$T(n) = \frac{n}{2} \log n + n \log n + n - 1 - \log n$$

$$\therefore T(n) = O(n \log n)$$

Best case:

Eg: Input: [4, 10, 3, 5, 1] Even if the array is sorted, it builds the heap and performs n operations, taking upto $O(\log n)$ times.

After building the max heap: [10, 5, 3, 4, 1]

$$\therefore \text{Best case} = O(n \log n)$$

Average case:

Eg: 3, 1, 5, 2, 4

- For a randomly arranged array, Heapsort builds the heap and repeated performs heapify, taking $O(\log n)$ for n elements.

$$\therefore \text{Average case} = O(n \log n)$$

Worst case: Eg: [5, 4, 3, 2, 1]

- Even in most unfavourable arrangement, Heapsort still performs heapify operations of $O(\log n)$ for n elements.

$$\therefore \text{Worst case} = O(n \log n)$$



Marwadi
University
Marwadi Chandarana Group



Faculty of Engineering & Technology

Department of CSE-AI&ML

Design and analysis of algorithm – 01AI0506

Conclusion :

Thus, the Max Heap Sort algorithm was successfully implemented and executed. The given array was sorted in ascending order using a max heap. The experiment helped in understanding the working of heapify and heap sort.

Name: Vemalatha Bhimireddy
Enrollment Number:92460118162